



Tutorial >

Using occurrences

Updated: 2018-09-06

Chronos introduces a powerful new scripting concept called Occurrences. An occurrence is basically a two-way event (forward and backward) scheduled to any point in time. It allows any piece of code to become easily rewindable without having to think of any time calculation.

Note that occurrences can also be one-way (forward only), for games that don't require rewind but would still like to take advantage of effortless planning. For example, no need to use a coroutine with `WaitForSeconds` to execute code in the future — just use `time.Plan(delay, YourMethod)`.

Using an occurrence, we will easily be able to despawn our enemies. However, you should first read the occurrence documentation and examples (it takes about 10 minutes) to better understand how occurrences work in simple scenarios. Click [here](#) to access the documentation:



Please read: [Occurrence Documentation & Examples](#)

Don't worry, we'll be waiting for you right here:

```
yield return WaitForFinishReading(documentation.occurrences);
```

Done reading? Cool. Now that you understand occurrences, let's change our enemies instantiation code from the spawner script. The line we want to change is this one:

```
Instantiate(enemies[enemyIndex], transform.position, transform.rotation);
```

Instead of this line, we want to instantly execute an occurrence that will instantiate the enemy, and destroy it when time rewinds to the current time. This occurrence will be repeatable, because if we let time flow normally after a rewind, we want that enemy to respawn at the same moment. Since this is a one-time piece of code in our game, there's no need to subclass the Occurrence class; we'll just use the delegates shortcut. Here is the framework:

```
time.Do // Instantly  
(  
    true, // Repeatable
```

```

delegate() // On forward
{
    // Spawn the enemy.
    // Must return a state transfer object.
    // In this case, we will return the spawned enemy.
},

delegate(object transfer) // On backward
{
    // Destroy the transferred spawned enemy.
    // The type of 'transfer' must be the same
    // type we return on forward.
}

);

```

The instantiation code stays exactly the same, except that we return the instantiated GameObject to be passed as transfer to the backward delegate. When rewinding, we'll use a simple Destroy on that GameObject.

```

time.Do
(
    true, // Repeatable

    delegate() // On forward
    {
        GameObject enemy = (GameObject)Instantiate(enemies[enemyIndex], transform.position);
        return enemy;
    },

    delegate(GameObject enemy) // On backward
    {
        Destroy(enemy);
    }

);

```

That's it! The final code for the spawner is below:

```

using UnityEngine;
using System.Collections;
using Chronos;

public class Spawner : BaseBehaviour
{
    public float spawnTime = 5f; // The amount of time between each spawn.
    public float spawnDelay = 3f; // The amount of time before spawning starts.
    public GameObject[] enemies; // Array of enemy prefabs.

    void Start ()
    {
        // Start calling the Spawn coroutine.
        StartCoroutine(Spawn());
    }

    IEnumerator Spawn ()

```

```

{
    yield return time.WaitForSeconds(spawnDelay); // Wait for the delay

    while (true) // Repeat infinitely
    {
        // Instantiate a random enemy.
        int enemyIndex = Random.Range(0, enemies.Length);

        time.Do
        (
            true, // Repeatable

            delegate() // On forward
            {
                GameObject enemy = (GameObject)Instantiate(enemies[enemyIndex], transf
                return enemy;
            },

            delegate(GameObject enemy) // On backward
            {
                Destroy(enemy);
            }
        );

        // Play the spawning effect from all of the particle systems.
        foreach(ParticleSystem p in GetComponentsInChildren<ParticleSystem>())
        {
            p.Play();
        }

        // Wait for the interval
        yield return time.WaitForSeconds(spawnTime);
    }
}

```



Don't forget to add a timeline to the spawner prefab!

We are now done with enemies and spawners. Take some time to play your awesome time-controlled platformer and stretch your legs. When you're done, we'll finish in style with one of the coolest features Chronos has to offer: area clocks.

Previous



Controlling time in script

Next

Adding an area clock →

Was this article helpful?

Be the first to vote!

 **Yes, helpful**

 **No, not for me**



PRODUCTS

LUDIQ

Contact

Blog

Copyright © 2022 Ludiq. All Rights Reserved. • • Terms of Service • Icons by IconMonstr and FatCow